

Welcome to CSCC63!

This week: Course introduction and definitions.

- I'll be your teacher this term — my name is Eric Corlett, and you can find my email at eric.corlett@utoronto.ca.
- My office hours will be held on Tuesdays from 3:00 to 4:30 in IC402. The TAs will also be holding office hours. Please come in if you have any questions about the course material. You can also send me an email if you need.
- Course information can be found on Quercus – in particular, we'll be using it for the course notes. The course syllabus can also be found there. The assignments will be submitted using Crowdmark.
- Let's take a few minutes to go over what's in the syllabus.

If you can do so at this moment, please go on to Quercus and have a look at the syllabus.

What is this course about?

This course is here to get you started in thinking about the *limits of computation*: what sort of problems can we reasonably compute using algorithms, and what problems will we never be able to fully answer? And if we can compute the answer to a problem, how efficiently can we do so?

There are a number of obvious candidates for questions we can't answer using computers and algorithms — e.g.,

What is your favourite colour?

What is the meaning of life?

Is there a god?

We won't even try to address these questions. Instead, we'll look for clear logical questions whose answers we still cannot find.

But We're always Building Better Computers...

Of course, we're designing more powerful computers all the time. So a statement about the limitations of your laptop today may not apply in a few years. In order to give limits that won't change, we have to talk about the limitations of logic itself.

We'll divide our questions about logic (and the course) between two main topics:

- **Computability:** What questions can logic ever solve?
- **Complexity:** What questions can logic solve efficiently?

Roughly speaking, the first half of the course will deal with the first topic, and the second half will deal with the second.

Decision Problems

In both topics, though, we'll need to have some way of talking about the problems we are likely to have to solve. A lot of what we do will relate to what we call **decision problems**.

A decision problem is a problem whose answer is either *Yes* or *No* (alternatively, *True* or *False*). We'll start by talking about these problems, and then extend our discussion to other types of problems later.

Some sample problems:

- Given integers x and y , what is $x^2 + y^3$?
(*not a decision problem*)
- Given integers x , y , and z , is $z = x^2 + y^3$?
(*decision problem*)
- Given integers x and z , is there an integer y such that $z = x^2 + y^3$?
(*decision problem*)

Decision Problems (2)

So you can see that we can ask many similar problems, some of which are decision problems, and some of which are not.



Important Idea:

Can you see the difference between the first two problems and the last? We'll return to this difference soon.

Of course, not all questions that we'll ask will be about numbers — we can also ask questions about graphs, geometric objects, or even other computer programs!

When talking about problems:

- We want to talk about problem inputs.
- We don't want to call them inputs, since inputs are for specific programs.

Remember that we're looking at problems and asking whether they can or can't be solved using programs. Using different terminology helps us keep the two levels separate.

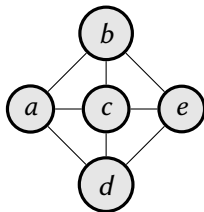
- So we call problem “inputs” **instances**.
- Inputs that result in a yes answer from a decision problem are *yes-instances*. *No-instances* are defined similarly.

Encoding Inputs

- We've said that we can ask questions about lots of different objects such as graphs or computer programs. These objects will then be part of our problem instances.
- We'll need to make one small limitation to the objects that we ask questions about, though — we have to be able to encode them into a form that we can put on a computer. So we won't ask questions about objects that have no reasonable encoding.
- Now, most objects we care about do have reasonable encodings, but things like real numbers (as opposed to their floating-point approximations) and arbitrary sets of natural numbers would take an infinite amount of memory, and so we won't work with them here.

Encoding Inputs (Example)

As an example, suppose we wanted to work with the graph



We can visualize graphs using pictures, but computers need something like an adjacency matrix or list:

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	1	1	0
<i>b</i>	1	0	1	0	1
<i>c</i>	1	1	0	1	1
<i>d</i>	1	0	1	0	1
<i>e</i>	0	1	1	1	0

We refer to the graph as G , and the **encoded format** for the input by using angle brackets, like so: $\langle G \rangle$. **Be careful about this distinction!**

Alphabets and Languages

Decision problems correspond nicely to the **languages** we talked about in CSCB36:

- An **alphabet** Σ is a finite set of characters. If we're dealing with computer inputs we often use the alphabet $\Sigma = \{0, 1\}$.
Recall that Σ^ is the set of finite strings over Σ .*
- A language over an alphabet Σ^* is a set of finite strings using the characters in Σ .

Languages are an alternative way of looking at decision problems:

- The yes-instances of a decision problem form a language.
- Given a language L , the question “Is x an element of L ?” is a decision problem.

What is an algorithm?

We should be clear about what we mean when we talk about *algorithms*.

- Intuitively: a sequence of logical (mechanical) steps we can use, or get a computer to use, to solve problem.
- But also: algorithms should halt on all possible inputs in a finite amount of time.

Definition: Algorithm

An *algorithm* to solve a problem is a logical sequence of steps which will, for any problem instance, terminate in a finite amount of time and give the solution for that instance.

So not all programs are algorithms — we can analyze those later, but this is a good starting point.

Algorithms as Mathematical Objects

How does all of this help us talk about the limitations of logic and computing?

- We need to start by fixing our terminology, which we've done. Then we need to figure out a general way of encoding algorithms.

If every algorithm can be encoded in a certain type of object, and that object can't do something, then algorithms can't do that thing either.

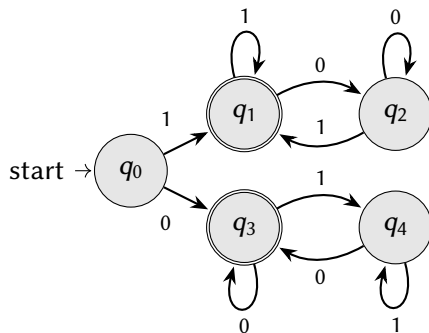
There's a lot of interesting history in how we decided to encode algorithms, and several contenders for object types:

- Alonzo Church's **Lambda Calculus**,
- Emil Post's computing machines,
- Alan Turing's **Turing Machines**
- ...

But they all have the same power! We'll look at Turing machines in this course.

OK, So What is a Turing Machine?

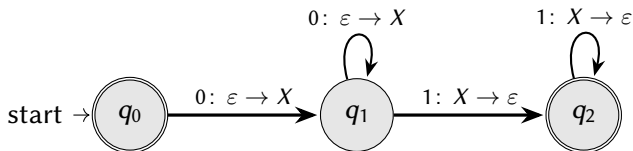
To start off, recall the DFSA's from CSCB36:



- A good start, but not very powerful — can't recognize 0^n1^n .

Can We Do Better?

What about CFGs/PDAs?



- Better — we can now recognize $0^n 1^n$.
- But not $\{w\#w \mid w \in \{0, 1\}^*\}$. We need to do a bit better.

The difference between DFSAs and CFGs is in their memory models.

What if we had unrestricted finite memory?

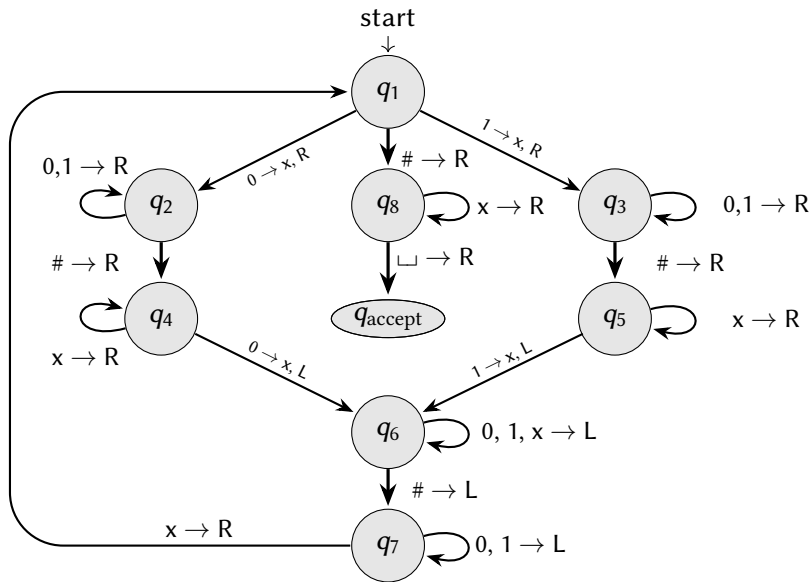
Turing Machines (TMs)

Like a DFSA, except:

- Instead of an input string, we use an input tape. Positions (cells) on this string represent memory cells.
- The input string is written on the left side of the tape. *We treat the string as having an unlimited number of blank $\square$ characters to its right.*
- In order to get unrestricted computing time, the head on the tape (the position of the character in question) can move either right or left upon reading a character. The direction of movement is determined by the arrow in the machine.
- We allow the TM to write on the tape. As before, it is the arrow on the diagram that tells us what to write.

How could we use this type of machine to recognize $\{w\#w \mid w \in \{0, 1\}^\}$?*

Don't Panic...



Try the input 010#011

- This is just like every other automaton: we start at state q_1 , and at the beginning of the tape.

We indicate this TM arrangement by writing $q_1010\#011$. You can see we've written the state and the tape. The position of the state on the string indicates the next character to be read.

This is called a **configuration**.

- When we read the first character, we follow its arrow, just like in the other automata.

We also perform the operations on the arrow: for a TM, we (optionally) write to the string and (not optionally) move the head left or right.

The arrow we choose is the $0 \rightarrow x, R$ arrow, since we've seen a 0. So we also write an x and move right. The new state is q_2 , so our next configuration is $xq_210\#011$.

Continuing the Idea:

Can you tell me what the next few configurations will be?

-
-
-

You'll notice on this last configuration we started moving left, not right. This is part of the extra computing power we can now use.

If you keep following this example, you'll eventually get to a point where there's no arrow to follow, so you'll reject (or alternatively, there'll be an arrow to a reject state).

If we reach the accept state, we accept. See what happens if you try plugging the string $1\#1$ into this machine.

TMs (Formal Definition):

We can define this TM as a tuple $M = (Q, \Sigma, \Gamma, \delta, s, q_A, q_R)$, where:

- Q is a set of states — the nodes in the above diagram.
- Σ is the input alphabet — this is the set of available input characters. In the example $\Sigma = \{0, 1, \#\}$. The blank symbol $\sqcup$ is never in Σ .
- Γ is the tape alphabet — this is the set of any characters we might see or use. In the example $\Gamma = \{0, 1, \#, x, \sqcup\}$. Note that $\Sigma \subseteq \Gamma$ and $\sqcup \in \Gamma$.
- δ is the transition function — the arrows on the diagram. Formally, it's a function

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\},$$

- s is the start state — it's the state q_1 in the example.
- q_A is the accept state — if we ever enter one of this state we halt and accept. In the example it's q_{accept} .
- q_R is the reject state — if we ever enter one of this state we halt and reject. It's implicit in the example.

Some Things to Remember:

- As we saw in the example, if we ever reach a point where there's no arrow out of our current state, we immediately go into an implicit reject state, reject, and halt.
- If we're on the left side of the tape and try to move left, we just stay in the same place.
- The blank spaces on the right side of the tape are implicit, so we don't need to write them. We do write spaces, though, if we've read them (e.g., in a configuration $x\#xq_8\sqcup$).

Why use TMs?

We'll actually spend a lot of time on the course looking at code — we won't spend too much time on the TMs themselves. But we like using them for a couple of reasons:

- We can basically program them the way we do regular computers.
- Their configurations are fairly well-behaved, and so we can do math on them. This will be what allows us to relate TMs (and computability results on TMs) to non-TM objects later on.

An Exercise

As an exercise, can we write a TM that recognizes the language

$$L = \{1^{2^n} \mid n \in \mathbb{N}\}?$$

Can we do Better?

TMs are good for describing algorithms because they can be “programmed” like computers.

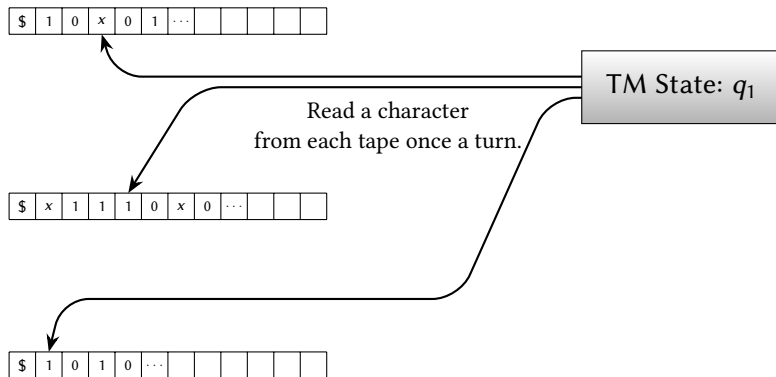
But is it possible to make even more powerful automata?

A (non-exhaustive) list of possible improvements:

- Use multiple tapes.
- Use non-determinism.
- Use RAM instead of linear memory.
- ...

Multi-tape Turing Machines

Like regular TMs, but with multiple tapes:



Emulating Multitape TMs

Idea: flatten the three tapes into one:

$$\begin{array}{l} \$10q_1x01\sqcup \\ \$x11q_110x0\sqcup \\ \$q_11010\sqcup \end{array}$$

into

$$\#\$10q_1x01\#\$x11q_110x0\#\$q_11010\sqcup$$

Notice: The tape head is in a different position on every string. In this construction the state is actually written on the tape — that way we can move the real tape head around without forgetting where we are on this particular tape.

Emulating Multitape TMs: A single step

In each step we:

- Start at the left hand side and move right to the end of the string.
- When we see a state, we look at the next character and remember it using TM states.
- When we reach the end of the string we move left again until we reach the beginning of the string.
- When we see a state, we update it.

This has a time cost — linear programs in a multitape TM can become quadratic, and so on. But solvable problems are still solvable.